VisionGuard Smart Cane

Step-by-step build guide: parts, assembly, wiring, code and testing



BACSE324 Embedded Systems Architecture, Course Project.
An Arduino Nano cane that beeps when an obstacle is near (faster as it gets closer) and gives a continuous tone when its tip touches water.

1. What the cane does

- **Obstacle warning:** an HC-SR04 ultrasonic sensor measures the distance ahead. Under 50 cm, the buzzer beeps, and the beeps get faster as the obstacle gets closer.
- **Water warning:** a moisture probe at the tip senses water or wet mud. The buzzer then sounds continuously. Water has the highest priority.
- **Portable:** one 18650 Li-ion cell with a rocker on/off switch.

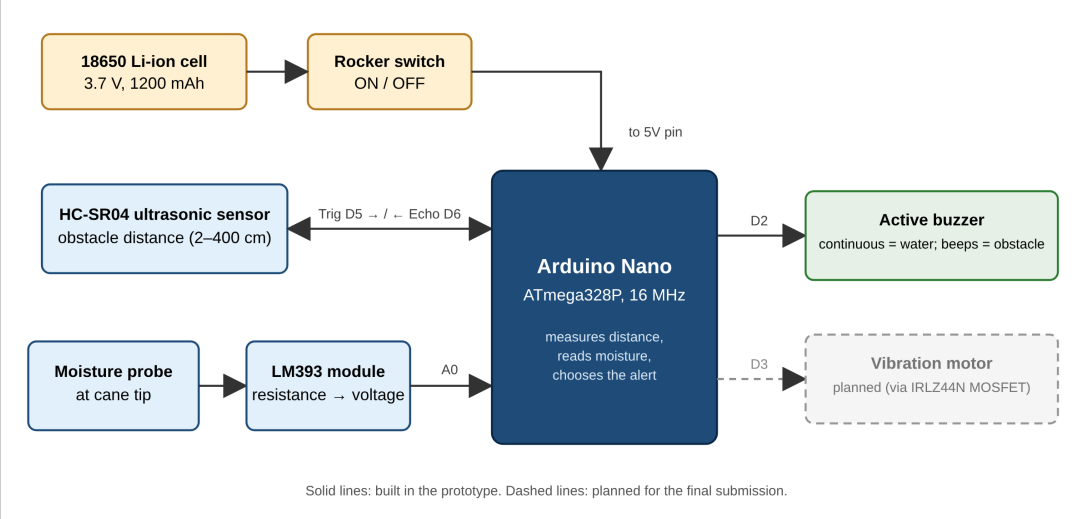


Figure 1. Block diagram. Solid lines are built; dashed lines are planned.

2. Parts and tools

#	Part	Specification	Job in the cane
1	Arduino Nano	ATmega328P, 16 MHz, 5 V	The "brain": reads the sensors, decides, drives the buzzer
2	HC-SR04 ultrasonic sensor	40 kHz, 2–400 cm	The "eyes": measures distance by timing a sound echo
3	Soil-moisture probe	Fork-type, two copper tracks	Senses water at the tip (water conducts electricity)
4	LM393 module	Pins A0, D0, GND, VCC + potentiometer	Turns the probe's resistance into a voltage on A0
5	Active buzzer	5 V, 2 pins	Makes the alert sound
6	18650 Li-ion cell + holder	3.7 V, 1200 mAh	Power
7	Rocker switch	6 A	Power on/off
8	Electrolytic capacitor	220 µF, 50 V	Smooths the supply so buzzer switching does not reset the Nano
9	PVC pipe	Cane length	Light, cheap cane body
10	Jumper wires, berg strips, single-strand wire	M-M, M-F, F-F	Connections

Tools: soldering iron (25 W), solder, flux, hot-glue gun, cable ties, breadboard (for testing), laptop with Arduino IDE, USB mini-B cable.

3. Circuit

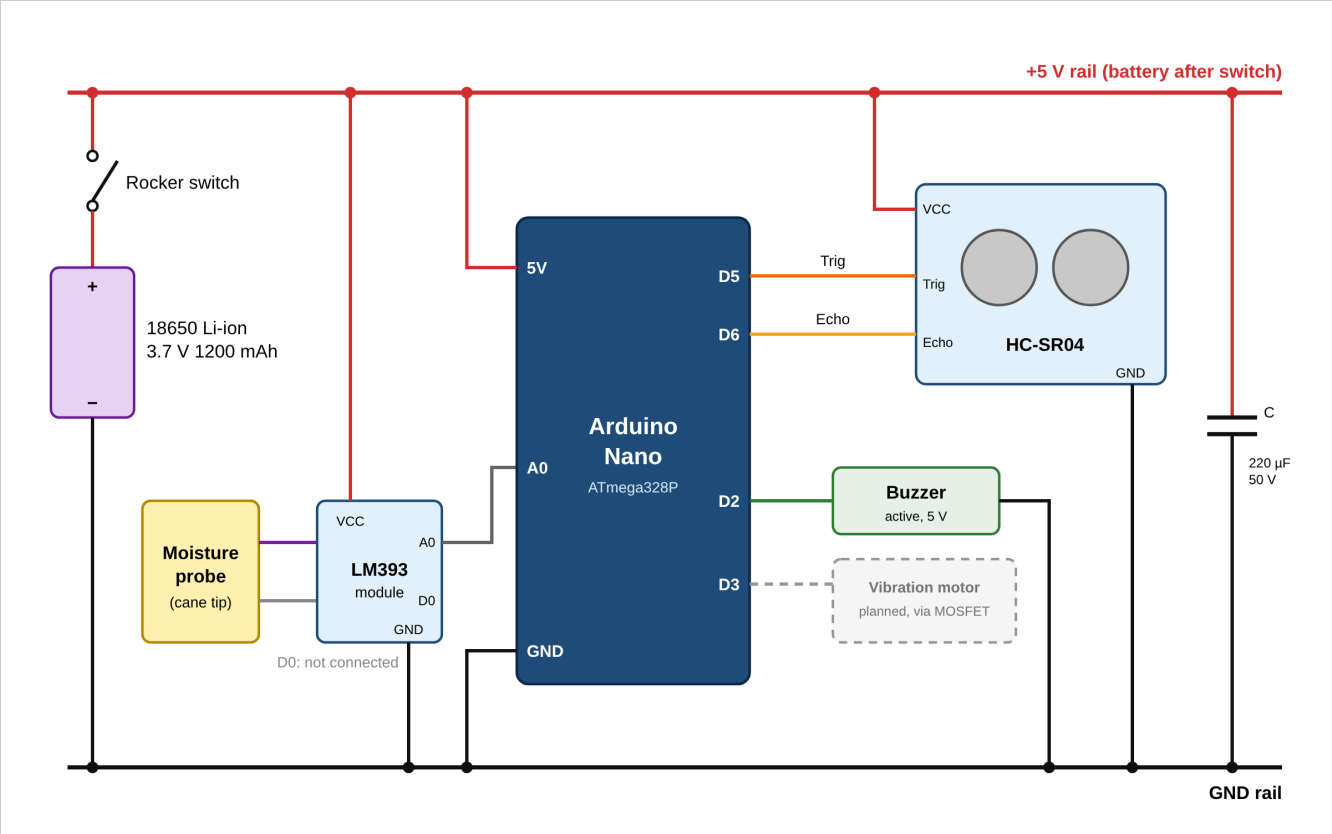


Figure 2. Circuit diagram. All modules share the +5 V and GND rails.

From	To (Nano)	Note
Battery + → rocker switch	5V	Direct to the 5V pin (not VIN, which needs about 7 V)
Battery −	GND	
HC-SR04 VCC / GND	5V / GND	
HC-SR04 Trig	D5	Nano sends the 10 µs trigger pulse
HC-SR04 Echo	D6	Nano measures the echo pulse width
LM393 VCC / GND	5V / GND	
LM393 A0	A0	Analog value 0–1023 (dry ≈ 1023, wet = lower)
LM393 D0	not connected	The threshold is set in code instead
Probe (2 wires)	LM393 probe pins	Either way round
Buzzer + (longer leg) / −	D2 / GND	
Capacitor + / − (stripe)	5V / GND	Polarity matters: the stripe side goes to GND

4. Assembly, step by step

Parts are placed from the bottom (ground) to the top (handle): **probe → LM393 → HC-SR04 → Nano + buzzer → switch → battery**. The heavier parts sit near the handle, which keeps the cane easy to swing.

Step 1: Test every part on the breadboard first

Before gluing or soldering anything, connect the Nano, HC-SR04, moisture module and buzzer on the breadboard with jumper wires, using the pin table above. Upload the code (Section 5) and check each part on the Serial Monitor. A faulty part is much easier to find here than on the finished cane.

Step 2: Prepare the cane body

Cut the PVC pipe to a comfortable cane length (roughly from the ground to the user's waist or chest). Clean the surface so the glue holds, and mark where each part will go.

Step 3: Fix the moisture probe at the tip

Hot-glue the probe to the bottom end of the pipe with the copper prongs pointing down, so they touch water first. Run the two probe wires (purple and grey) up along the pipe to where the LM393 module will sit.

Keep the glue on the black part of the probe only. The copper tracks must stay exposed.



Probe at the cane tip

Step 4: Mount the LM393 module

Glue the blue module onto the pipe above the probe. Plug the probe wires into its two-pin side. On the four-pin side, connect:

- VCC (red) → 5V rail
- GND (brown) → GND rail
- A0 (grey) → Nano A0
- D0 → leave empty



LM393 module: A0, GND and VCC used; D0 empty

Step 5: Mount the ultrasonic sensor facing forward

Stick the HC-SR04 on a foam block so it stands clear of the pipe and the two "eyes" (T = transmitter, R = receiver) face forward in the walking direction. Connect its four pins with female jumpers:

- VCC → 5V, GND → GND
- Trig → D5, Echo → D6

Nothing should block the front of the sensor, including wires and glue.



HC-SR04 on a foam block

Step 6: Mount the Nano and connect the wires

Hot-glue the Nano onto the pipe with its USB port reachable, so it can be reprogrammed later. Solder male header pins onto the Nano. Plug the sensors' female jumper wires onto these pins: orange (Trig) → D5, yellow (Echo) → D6, grey (moisture) → A0. Solder the power wires: all the 5V wires join at the Nano's 5V pin and all the GND wires at its GND pin.

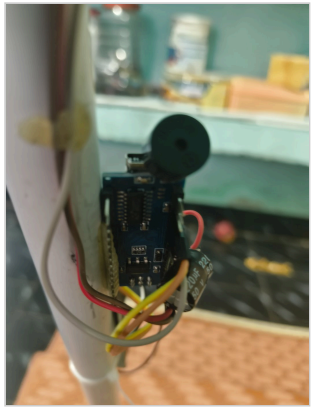
> **Soldering tips:** heat the pin and the wire together for 2–3 seconds, then feed in the solder. A good joint is shiny and cone-shaped. Do not let solder bridge two neighbouring pins. Cover bare joints with tape or heat-shrink.

Step 7: Fit the buzzer

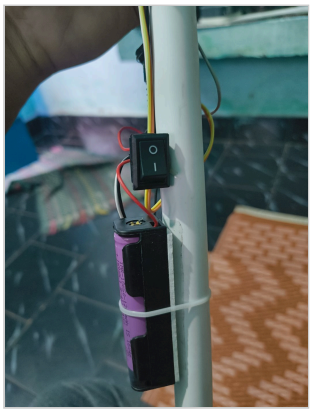
Solder the buzzer's **+ (longer leg)** to D2 and the other leg to GND. Fix it next to the Nano with glue.

Step 8: Power circuit

1. Fix the battery holder near the handle with a cable tie.
2. Glue the rocker switch where the thumb can reach it.
3. Battery **+** (red) → one switch terminal; other switch terminal → Nano **5V**.
4. Battery **−** (black) → Nano **GND**.
5. Solder the 220 µF capacitor across 5V and GND on the Nano, with the stripe (−) to GND.



18650 cell, cable-tied



Rocker switch between battery + and the 5V pin

Safety: double-check the polarity before inserting the battery. Reversed + and − can destroy the Nano. Never let bare battery wires touch each other, because a shorted Li-ion cell gets very hot. Charge the cell only with a proper Li-ion charger.

5. Uploading the code

1. Install the **Arduino IDE 2.x**. If no port appears for a clone Nano, install the **CH340 driver**.
2. **Switch the cane OFF**, then plug the Nano into the laptop with the USB cable.
3. Tools → Board → **Arduino Nano**; Processor → **ATmega328P** (try *Old Bootloader* if the upload fails); Tools → Port → the new COM port.
4. Open `visionguard.ino` and click **Upload**.
5. Open Tools → Serial Monitor at **9600 baud** to see live readings.

> The battery connects straight to the 5V pin. **Keep the rocker switch OFF while USB is plugged in**, otherwise the laptop's 5 V flows back into the Li-ion cell.

How the code works

Part of the code	What it does
`#define TRIG_PIN 5` …	Gives each pin a name, so the code matches the wiring
`OBSTACLE_DISTANCE_THRESHOLD = 50`	Alert distance in cm
`WET_SOIL_THRESHOLD = 500`	Moisture readings below this count as water
`readDistanceCm()`	Sends a 10 µs pulse on Trig, measures how long Echo stays HIGH with `pulseIn` (30 ms timeout), then distance = time × 0.034 ÷ 2
`analogRead(SOIL_PIN)`	Reads 0–1023 from the moisture module
`if (soilMoisture < 500)`	Water: buzzer ON continuously (top priority)
`else if (distance < 50)`	Obstacle: a 50 ms beep, then a gap of 50–400 ms from `map()` (closer = shorter gap = faster beeps)
`else`	Path clear: buzzer OFF
`delay(30)`	Short pause so the old echo dies out before the next ping

Why 0.034 and ÷ 2? Sound travels about 340 m/s = 0.034 cm per microsecond. The pulse goes to the object and back, so the time is halved.

Full program

```
// VisionGuard Smart Cane - obstacle + wet-surface warning
// Board: Arduino Nano (ATmega328P)
//
// PROVISIONAL: based on the original sketch in
// firmware/original/visionguard_original. Same pins, same thresholds,
// same alert priority. Pin numbers are taken from that sketch and still
// have to be confirmed on the real wiring (see docs/REVIEW1_PREP.md).
//
// Changes from the original:
//   1. pulseIn() has a 30 ms timeout, so a missed echo no longer freezes
//      the loop for 1 second (the default timeout).
//   2. Distance is clamped before map(), so the beep gap always stays
//      between 50 ms and 400 ms.
//   3. Serial Monitor prints the alert state, which helps during the demo.
```

```
// Pin Configurations
#define TRIG_PIN 5
#define ECHO_PIN 6
#define SOIL_PIN A0
#define BUZZER_PIN 2   // Active buzzer (beeps when the pin is HIGH)
#define VIB_PIN 3      // Vibration output - no motor confirmed in the hardware yet.
                       // Never connect a motor straight to this pin; use the
                       // IRLZ44N MOSFET + flyback diode as a driver.

// Calibration Thresholds
const int OBSTACLE_DISTANCE_THRESHOLD = 50; // Distance in cm
const int MIN_DISTANCE = 2;                 // HC-SR04 cannot measure closer than ~2 cm
const int WET_SOIL_THRESHOLD = 500;         // Adjust based on sensor readings (Lower = Wetter)
const unsigned long ECHO_TIMEOUT_US = 30000; // ~5 m round trip; longer means "nothing in range"

void setup() {
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(VIB_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(VIB_PIN, LOW);
}

// Returns distance in cm, or 0 if no echo came back in time
int readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = pulseIn(ECHO_PIN, HIGH, ECHO_TIMEOUT_US);
  // Sound travels 0.034 cm/us; divide by 2 because the pulse goes out and back
  return duration * 0.034 / 2;
}

void alertsOff() {
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(VIB_PIN, LOW);
}

void loop() {
  // 1. Read Distance from Ultrasonic Sensor
  int distance = readDistanceCm();

  // 2. Read Soil Moisture Level (dry ~1023, wet = lower)
  int soilMoisture = analogRead(SOIL_PIN);

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.print(" cm | Soil Value: ");
  Serial.print(soilMoisture);

  // 3. Alert Logic
  if (soilMoisture < WET_SOIL_THRESHOLD) {
    // Priority 1: Water/Wet Mud Detected - Continuous Warning
    Serial.println(" | ALERT: WATER");
    digitalWrite(BUZZER_PIN, HIGH);
    digitalWrite(VIB_PIN, HIGH);
  }
  else if (distance > 0 && distance < OBSTACLE_DISTANCE_THRESHOLD) {
    // Priority 2: Obstacle Detected - closer obstacle = faster beeping
    Serial.println(" | ALERT: OBSTACLE");
    int clamped = constrain(distance, MIN_DISTANCE, OBSTACLE_DISTANCE_THRESHOLD);
    int delayTime = map(clamped, MIN_DISTANCE, OBSTACLE_DISTANCE_THRESHOLD, 50, 400);
```

```
    digitalWrite(BUZZER_PIN, HIGH);
    digitalWrite(VIB_PIN, HIGH);
    delay(50); // Sharp beep
    alertsOff();
    delay(delayTime); // Dynamic wait interval
  }
  else {
    // Path Clear
    Serial.println(" | CLEAR");
    alertsOff();
  }

  delay(30);
}
```

6. Testing and calibration

Test	How	Expected result
Power	Switch ON (no USB)	Nano PWR LED lights up
Path clear	Point the sensor at open space	Silent; Serial shows `CLEAR`
Obstacle	Move a notebook slowly from 60 cm to 5 cm	Beeping starts under 50 cm and gets faster as it comes closer
Water	Dip the probe tip in a cup of water	Continuous tone; Serial shows `ALERT: WATER`
Priority	Tip in water and notebook close	Continuous tone (water wins)

Calibrating the water threshold

1. On the Serial Monitor, note the "Soil Value" with the probe **dry** (for example ~1020) and **in water** (for example ~350).
2. Set `WET_SOIL_THRESHOLD` about halfway between them (for example 650) and upload again.

7. Troubleshooting

Problem	Likely cause	Fix
Upload fails, "not in sync"	Wrong bootloader setting	Processor → ATmega328P (Old Bootloader)
No COM port	Missing CH340 driver or a charge-only cable	Install the driver; try a different cable
Distance always 0	Trig/Echo swapped or loose	Check D5 = Trig, D6 = Echo
Buzzer always ON	Probe wet or threshold too high	Dry the probe; recalibrate the threshold
Buzzer silent	Buzzer reversed or passive type	Swap the legs; an active buzzer is required
Nano keeps resetting	Weak battery or voltage dips	Charge the cell; check the capacitor
Readings jump around	Soft or angled targets, loose wires	Test on a flat, hard surface; reseat the jumpers

8. Limitations and next improvements

- Single 3.7 V cell on the 5V pin with no regulation → add a **5 V boost converter**, a **TP4056 charger** and the **1 A fuse**.
- Buzzer is hard to hear in traffic → add a **vibration motor** driven by the **IRLZ44N MOSFET** with a flyback diode.
- The sensor cone (~15°) misses head-level obstacles and drop-offs → add sensors angled up and down.
- The resistive probe corrodes and senses water only on contact → use a **capacitive** sensor.
- 50 cm gives little warning time → test and raise it to about 100 cm.